Bench wiring sheet - protocol emulator on the MachXO3L kit (3 Nov 2026)

One wire per row: **FROM** one end **TO** the other, **AND** a third end where the wire has one (a pull-up, a second board). Kit pins are written **J4-n (ball)**: n is the numbered hole on header J4, the ball name (K12...) is the FPGA's own label printed beside it, the same connection under a second name, for checking by continuity. **Engine pin** numbers (0-19) are what the chip's programs call that pin. Everything is **3.3 V logic** (FPGA and ESP32 pins are not 5 V tolerant); one common ground between every board.

1. Core: kit to ESP32 host. Fit these eight first, then rerun.ps1 (command r): ID reads 0x0022, 0 problems.

FROM	TO	AND (third end)	What it is
J4-3 (K12)	ESP32 GPIO 5		CS, host SPI
J4-5 (M14)	ESP32 GPIO 23		MOSI, host SPI
J4-7 (L14)	ESP32 GPIO 19		MISO, host SPI
J4-9 (M15)	ESP32 GPIO 18		SCK, host SPI
J4-11 (GND)	ESP32 GND	every other board's GND	common ground
J4-4 (K13)	ESP32 GPIO 4		IRQ: high while any engine has data
J4-6 (N14)	ESP32 GPIO 16		engine pin 0: engine A's UART out, into ESP32 Serial2 RX
J4-26 (G15)	ESP32 GPIO 17		engine pin 15: the ESP32 sends test pulses into the chip (glitch filter)

J4-1 and J4-2 are the kit's 3.3 V: use for pull-ups only, never join them to the ESP32's 3.3 V. Kit: **bash fpga/machxo3/build.sh load** (volatile SRAM, never the NVCM). ESP32 (the scripts find its port themselves): **esp32\upload_and_watch.ps1 -Sketch chip_test**.

2. I2C bus (engine pins 2 = SCL, 3 = SDA). ESP32 as the slave: rerun.ps1 -Cmd i. UNO R3 as the slave: -Cmd n.

FROM	TO	AND (third end)	What it is
J4-10 (M16) SCL	ESP32 GPIO 22 or UNO A5	5.1 k resistor to 3.3 V	SCL, engine pin 2
J4-13 (L15) SDA	ESP32 GPIO 21 or UNO A4	5.1 k resistor to 3.3 V	SDA, engine pin 3
UNO GND	common ground		only when the UNO is the slave; its own 5 V pull-ups are switched off by its sketch
3.3 V for the two pull-ups	J4-1 or the UNO's 3.3 V pin		one source, not both

Never both slaves at once (both answer at address 0x50): unplug the UNO's A4 and A5 for -Cmd i. UNO sketch: arduino/i2c_slave_nano on COM11.

3. SPI master to the 74HC595 (engine pins 4 to 7). rerun.ps1 -Cmd s.

FROM	TO	AND (third end)	What it is
J4-14 (L16)	595 pin 11 SRCLK		SCK, engine pin 4
J4-15 (K14)	595 pin 14 SER		MOSI, engine pin 5
J4-16 (K16)	595 pin 9 QH'		MISO, engine pin 6: the 595's serial out comes back
J4-17 (K15)	595 pin 12 RCLK		CS / latch, engine pin 7
595 pin 16 VCC	3.3 V		power
595 pin 8 GND	common ground		power
595 pin 13 OE	GND		outputs enabled
595 pin 10 SRCLR	3.3 V		not clearing
595 pins 15, 1, 2, 3, 4, 5, 6, 7 (QA..QH)	LED long leg	LED short leg -> 330 ohm -> GND	eight LEDs; the run ends on 0x55 so QA (pin 15) is high

Before the rebuild the LEDs stayed dark with the chip side proven good: meter pins 16, 8, 13, 10 first, then pin 15 after a run, then the QH' lead.

4. Every J4 hole the chip uses (for anything new)

J4 holes	Balls	Engine pins	Direction
6, 8, 10, 13, 14, 15, 16, 17	N14, N16, M16, L15, L16, K14, K16, K15	0, 1, 2, 3, 4, 5, 6, 7	in and out (open drain able)
18, 19, 20	J14, H14, J15	8, 9, 10	out only
23, 24, 25	J16, H15, H16	11, 12, 13	out only
26, 27, 28, 29, 30	G15, G16, F15, F16, E15	15, 16, 17, 18, 19	in only
1, 2	3.3 V		
11, 12	GND		

Holes and balls pair up in order: hole 6 is ball N14 (engine pin 0), hole 8 is N16 (pin 1), and so on along each row. Kit LEDs: P16 lights for IRQ, N15 is the heartbeat (they light when driven low).
Cyclone IV EP4CE6 (arrived 3 Nov): pin locations still to be chosen in fpga/cyclone4/chip_c4.qsf; the same eight leads of section 1 move across; set CHIP_HZ = 50e6 in the sketch (SPI goes to 4 MHz).
If it doesn't answer: esp32 rerun.ps1 -Cmd p (lead probe: a FLOATING pin = lead open or wrong hole; ID 0x0000 with MISO driven = CS/SCK/MOSI not arriving; 0xFFFF = MISO open), -Cmd q (IRQ held high 15 s), -Cmd l (repeating ID reads for the oscilloscope). Writes working but reads all zero = MISO lead's J4 end in a hole held low (11/12 GND, 18-25 driven low). The kit forgets its build on any power blip: reload with bash fpga/machxo3/build.sh load.

Order of work after replugging

1. Section 1 only. Load the kit, upload the sketch, rerun.ps1 (command r): ID 0x0022, 0 problems. 2. Add section 2: rerun.ps1 -Cmd i (ESP32 slave) or -Cmd n (UNO slave). 3. Add section 3: rerun.ps1 -Cmd s.